CHSM Reference Manual

C Hierarchical State Machine
Editor & Runtime Framework

Version 1.0.0

2026-04-12

Janos Szeman

Table of Contents

Contents

1 Introduction

1.1 What is CHSM?

CHSM (C Hierarchical State Machine) is an integrated toolchain for designing, generating, and executing UML-style hierarchical state machines on embedded targets.

The project consists of three layers:

1. **Visual Editor** — A browser-based GUI for drawing states and transitions.
2. **Code Generator** — Jinja2-based templates that produce compileable code for C, Python, JavaScript, Java, and VHDL.
3. **CRF Runtime Library** — A zero-malloc, ISR-safe active-object framework written in C for bare-metal and RTOS targets.

1.2 Key Features

Feature	Description
Visual Editor	Browser-based drag-and-drop state machine designer with theme support.
Multi-Language	Generate C, Python, JavaScript, Java, and VHDL from a single model.
Zero Malloc	All queues and pools are statically pre-allocated — no `malloc` at runtime.
ISR-Safe	Event posting uses atomic compare-and-swap; safe to call from interrupts.
Configurable	Three flags (`CHSM_CFG_LITE`, `NO_FAULT_CNT`, `NO_DEBUG`) trade features for smaller footprint.
Debug Channel	TCP-based real-time state monitoring from the GUI editor.
Module Wizard	Cookiecutter scaffolding generates build-ready module directories.

1.3 Document Scope

This reference manual covers:

- System architecture and design principles.
- Complete C API reference for all runtime structs, functions, and macros.
- Code generation pipeline and template customisation.
- Build system and runtime configuration options.

For quick-start tutorials, see the HTML documentation built with Sphinx.

2 Architecture

2.1 Layer Model

The CHSM system is organised into three layers. Each layer communicates strictly downward — the runtime has no dependency on the GUI.

Layer	Responsibility
GUI Editor	Visual state/transition editing, JSON model persistence, theme management, debug panel.
Code Generator	Parse JSON model → build AST → render Jinja2 templates → output source files.
CRF Runtime	Run-to-completion event loop, static memory pools, atomic queues, garbage collection.

2.2 Execution Model

The CRF runtime uses a **run-to-completion** (RTC) execution model:

1. `crf_step()` iterates over all registered state machines.
2. For each HSM, it dequeues one event from the event queue.
3. The event is dispatched to the current state handler.
4. If the handler returns `C_RES_PARENT`, the event propagates up the hierarchy.
5. If the handler returns `C_RES_TRANS`, exit and entry functions are called in order.
6. After dispatch, `crf_gc()` decrements the reference count and frees the event if it reaches zero.

No preemption occurs within a single dispatch cycle. This guarantees that state handlers execute atomically with respect to other events.

2.3 Struct Relationships

The core data structures form a clear ownership hierarchy:

- **`crf_tst`** owns the array of state machines and memory pools.
- **`chsm_tst`** embeds two `cqueue_tst` instances (event queue and defer queue).
- **`cpool_tst`** manages a contiguous buffer of fixed-size event blocks.
- **`cevent_tst`** is the base event type with signal and GC metadata.
- All derived events embed `cevent_tst` as their first member.

2.4 Thread Safety Model

Operation	Safety	Notes
`cqueue_put`	ISR-safe	Atomic CAS on head; multiple producers allowed.
`cqueue_get`	Main only	Single consumer assumed.
`cpool_new`	ISR-safe	CAS-based free-list pop.
`cpool_gc`	ISR-safe	CAS-based free-list push.
`crf_step`	Main only	Iterates HSMs sequentially in a single thread.

2.5 Directory Layout

```
chsm/
├── gui/                    # Visual editor + code generators
│   ├── chsm_backend.py    # Eel backend + debug server
│   ├── c_gen/              # C code generator
│   ├── py_gen/             # Python code generator
│   ├── js_gen/             # JavaScript code generator
│   ├── java_gen/           # Java code generator
│   ├── vhdl_gen/           # VHDL code generator
│   └── web/                # Frontend HTML/CSS/JS
├── languages/
│   └── c/                  # C runtime + modules
│       ├── c_rf/           # CRF library (cevent, cqueue, cpool, crf)
│       ├── modules/        # HSM modules (chsm core + applications)
│       ├── cmake_utils/    # CMake helper macros
│       └── unity/          # Unity test framework
└── docs/
    ├── sphinx/             # Sphinx documentation (HTML/PDF)
    └── typst/              # Typst documentation (PDF)
```

3 C API Reference

This chapter documents every public struct, function, and macro in the CRF runtime library.

3.1 `cevent` — Event Base

3.1.1 `cevent_tst`

Base event structure. All derived events must embed this as their first member.

```
typedef struct cevent_tst {
    signal_t    sig;       // event type identifier
    gc_info_tst gc_info;   // pool id + reference count
} cevent_tst;
```

Size: 4 bytes (with default `signal_t = uint16_t`).

3.1.2 `gc_info_tst`

```
typedef struct {
    uint16_t ref_cnt : 12;   // 0 – 4095
    uint16_t pool_id :  4;   // 0 – 15
} gc_info_tst;
```

- **ref_cnt** — Incremented on queue insertion, decremented after dispatch. Zero triggers deallocation.
- **pool_id** — Identifies the source pool. Pool 0 (`CEVENT_INVALID_POOL`) marks static events.

3.1.3 Signal Classes

Constant	Value	Purpose
`CEVENT_INVALID_POOL`	0	Static event marker.
`CRF_SIGNAL_CLASS_MOD_INTERNAL`	1	Module-internal signals.
`CRF_SIGNAL_CLASS_START`	2	First application class.
`CRF_SIGNAL_CLASS_SIZE`	100	Signals per class.

Macro: `SIGNAL_FROM_CLASS(CLASS)` ⟶ `CLASS × 100`.

3.1.4 Functions

- `cevent_ref_cnt_inc(e)` — Increment reference count (no-op for static events).
- `cevent_ref_cnt_dec(e)` — Decrement reference count (no-op for static events).

3.2 `cqueue` — Event Queue

3.2.1 `cqueue_tst`

Circular FIFO of event pointers. Capacity must be a power of two.

Key fields:
- `events` — Pre-allocated pointer array.
- `max` — Capacity.
- `head`, `tail` — Atomic read/write indices.
- `mask` — `max - 1` for efficient wrapping.
- `fault_cnt` — Overflow counter (omitted with `CHSM_CFG_NO_FAULT_CNT`).

3.2.2 Functions

Function	Returns	Description
`cqueue_init(q, events, max)`	`int32_t`	Initialise queue. Returns 0.
`cqueue_put(q, e)`	`int32_t`	Enqueue right. ISR-safe. Returns −1 on overflow.
`cqueue_put_left(q, e)`	`int32_t`	Enqueue left (priority). ISR-safe.
`cqueue_get(q)`	`cevent*`	Dequeue left. Main-thread only. NULL if empty.
`cqueue_get_right(q)`	`cevent*`	Dequeue right. Used by `chsm_recall`.

3.2.3 Dispatch Macros

`CQUEUE_PUT`, `CQUEUE_PUT_LEFT`, `CQUEUE_GET`, `CQUEUE_GET_RIGHT` — resolve to function-pointer calls normally, or direct calls with `CHSM_CFG_LITE`.

3.3 `cpool` — Memory Pool

3.3.1 `cpool_tst`

Lock-free fixed-size block allocator using atomic CAS.

Key fields:
- `pool` — Byte buffer.
- `esize` — Bytes per block.
- `ecnt` — Total blocks.
- `head` — Atomic offset to first free block.

3.3.2 Functions

Function	Returns	Description
`cpool_init(p, buf, size, cnt)`	void	Thread free-list through buffer.

Function	Returns	Description
`cpool_new(p)`	`void*`	CAS pop. ISR-safe. NULL if exhausted.
`cpool_gc(p, e)`	`bool`	CAS push. ISR-safe. False if wrong pool.

Dispatch macros: `CPOOL_NEW(p)`, `CPOOL_GC(p, e)`.

3.4 `chsm` — State Machine

3.4.1 `chsm_tst`

State machine instance. Embeds two queues (event + defer), a state handler pointer, a send callback, and a linked-list next pointer.

3.4.2 Result Enum

Value	Meaning
`C_RES_HANDLED`	Event processed.
`C_RES_TRANS`	State transition triggered.
`C_RES_PARENT`	Delegate to parent state.
`C_RES_IGNORED`	Event intentionally skipped.
`C_RES_GUARDS`	Guard conditions — none true.

3.4.3 Functions

Function	Description
`chsm_ctor(self, init, events, qlen, dlen)`	Construct HSM with initial state and queue buffers.
`chsm_init(self)`	Send `C_SIG_INIT` and execute initial transition.
`chsm_dispatch(self, e)`	Dispatch event to current state; propagate on `C_RES_PARENT`.
`chsm_defer(self, e)`	Move event to defer queue.
`chsm_recall(self, e)`	Move all deferred events to front of event queue.

3.4.4 Inline Helpers

- `chsm_transition(self, target)` — Record transition, return `C_RES_TRANS`.
- `chsm_handled(self)` — Return `C_RES_HANDLED`.
- `chsm_ignored(self)` — Return `C_RES_IGNORED`.

3.5 `crf` — Reactive Framework

3.5.1 `crf_tst`

Top-level framework. Owns the HSM array and pool array. Provides the event loop.

3.5.2 Functions

Function	Description
`crf_init(self, hsms, pools, cnt)`	Initialise framework with HSM and pool arrays.
`crf_new_event(self, size, sig)`	Allocate event from matching pool. NULL on failure.
`crf_publish(self, e)`	Post event to all registered HSMs.
`crf_post(self, e, q)`	Post event to specific queue.
`crf_step(self)`	Process one event per HSM. Returns true if work done.
`crf_gc(self, e)`	Decrement ref_cnt; free to pool if zero.

3.5.3 Convenience Macros

Macro	Purpose
`CRF_NEW(SIGNAL)`	Allocate event by signal name.
`CRF_POST(e, q)`	Post event to queue.
`CRF_EMIT(e)`	Send via active HSM's callback.
`CRF_STEP()`	Run one event cycle.
`CRF_GC(e)`	Garbage-collect event.
`CRF_SIG_VAR(SIG, var, e)`	Declare and cast event pointer.
`CRF_POST_TO_SELF(e)`	Post to own queue with priority.

3.6 `chsm_cfg` — Build Configuration

Three compile-time flags control the runtime footprint:

Flag	Effect
`CHSM_CFG_LITE`	Replace function-pointer dispatch with direct calls. Enables inlining.
`CHSM_CFG_NO_FAULT_CNT`	Remove `fault_cnt` from queues (−4 B each).
`CHSM_CFG_NO_DEBUG`	Strip debug instrumentation (−20 B per module).

Define in `chsm_cfg.h` (included before all CRF headers) or pass as compiler flags.

4 Code Generation

4.1 Pipeline

The code generator transforms a visual model into compileable source files through three stages:

1. **JSON Model** — The `.chsm` file saved by the visual editor.
2. **Parser / AST** — `parser.py` walks the JSON, resolves parent chains, computes exit/entry paths, and builds a flat AST dictionary.
3. **Jinja2 Rendering** — Language-specific `.j2` templates consume the AST and produce output files.

4.2 Supported Languages

Language	Generator	Template
C	`gui/c_gen/hsm/sm.py`	`sm_temp.c.j2`, `sm_temp.h.j2`
Python	`gui/py_gen/render.py`	`state_machine_template.py.j2`
JavaScript	`gui/js_gen/render.py`	`state_machine_template.js.j2`
Java	`gui/java_gen/render.py`	`state_machine_template.java.j2`
VHDL	`gui/vhdl_gen/render.py`	`state_machine_template.vhdl.j2`

4.3 AST Structure

The parser produces a dictionary with the following top-level keys:

- **name** — Module name (string).
- **states** — List of state objects with `name`, `id`, `parent`, `entry`, `exit`, `children`.
- **transitions** — List with `source`, `target`, `signal`, `guard`, `actions`, plus pre-computed `exit_path` and `entry_path`.
- **signals** — Deduplicated list of signal names.

The `exit_path` and `entry_path` fields are the critical output of the parser: they contain the ordered list of user functions (exit bottom-up, entry top-down) that must execute during a transition. The runtime does not compute these at run time.

4.4 C Generator Extras

The C generator produces additional files beyond the state machine skeleton:

- **Function stubs** (`sm_functions_temp.c.j2` / `.h.j2`) — Skeleton implementations of entry, exit, guard, and action functions. These are generated once and preserved across regeneration.
- **Unit test scaffold** (`unittest_temp.c.j2`) — A Unity test file with one test case per transition.

4.5 Adding a Language

To add a new target language:

1. Create `gui/<lang>_gen/` directory.
2. Implement `render.py` with a `render(ast, output_path)` function.
3. Create a Jinja2 `.j2` template.
4. Register the language in the editor's Generate menu.

4.6 CLI Usage

All generators can be invoked from the command line without the GUI:

```
python -m gui.c_gen.hsm.sm    model.json --output build/
python -m gui.py_gen.render    model.json
python -m gui.js_gen.render    model.json
python -m gui.java_gen.render  model.json
python -m gui.vhdl_gen.render  model.json
```

5 Configuration

5.1 CMake Build System

The C runtime uses CMake with helper macros from `cmake_utils/cmake_utils.cmake`.

5.1.1 Key Macros

- **`add_module_lib`** — Create a static library from module sources with correct includes and link deps.
- **`add_module_test`** — Create a Unity test executable linked to the module under test.

5.1.2 Build Commands

```
cd languages/c
cmake -B build -S . -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure
```

5.1.3 Cross-Compilation

```
cmake -B build -S . \
    -DCMAKE_TOOLCHAIN_FILE=cmake_utils/compilers/arm-none-eabi.cmake \
    -DCMAKE_BUILD_TYPE=MinSizeRel
```

5.2 Runtime Tuning

5.2.1 Profiles

Profile	Flags	Use Case
Development	*(none)*	Full diagnostics, debug logging, fault counters.
Release	`CHSM_CFG_LITE`	Reduced RAM; overflow diagnostics retained.
Minimal	`LITE + NO_FAULT_CNT + NO_DEBUG`	Smallest footprint (≤ 8 KB RAM MCUs).

5.2.2 RAM Savings (Cortex-M, 32-bit)

Config	Queue ×2	Pool	Framework
Default	56 B	20 B	48 B
`CHSM_CFG_LITE`	24 B	10 B	12 B
`+ NO_FAULT_CNT`	16 B	10 B	12 B
`+ NO_DEBUG`	16 B	10 B	12 B

Total savings with all three flags: 86 bytes per HSM system.

5.2.3 Queue Sizing

- **Event queue**: 8–16 entries (power of 2). Covers ISR burst between `crf_step` calls.
- **Defer queue**: 4 entries. Rarely more than 2–3 deferred simultaneously.

5.2.4 Pool Sizing

1. Identify the largest derived event type.
2. Count peak in-flight events (queued + deferred + dispatching).
3. Add 2–4 blocks margin.

5.3 Project Settings

5.3.1 `settings.json`

Code generation defaults stored at `gui/c_gen/templates/settings.json`:

Key	Type	Description
`output_dir`	string	Relative path for generated files.
`language`	string	Default target: `c`, `python`, `js`, `java`, `vhdl`.
`indent`	int	Indentation width (spaces).
`line_ending`	string	`lf` or `crlf`.
`header_guard_style`	string	`pragma_once` or `ifndef`.

5.3.2 Themes

CSS files in `gui/web/themes/`. Create a custom theme by copying `default.css` and editing CSS custom properties (`--bg-primary`, `--accent`, `--state-fill`, etc.).